

TEORÍA DE LENGUAJES DE PROGRAMACIÓN

Motor de Inferencia Lógica como Servicio

Reporte del Proyecto Final

Descripción: Servicio web que ejecuta consultas sobre una base de conocimiento en Prolog mediante una API REST en Node.js.

Tecnologías: Node.js · Express · Tau Prolog

Repositorio: Proyecto-TLP

Materia: Teoría de Lenguajes de Programación

1. Introducción

El presente proyecto consiste en el diseño e implementación de un **Motor de Inferencia Lógica como Servicio**: un sistema web que permite ejecutar consultas sobre una base de conocimiento escrita en **Prolog** a través de una API REST desarrollada en **Node.js** con el framework **Express**.

El sistema funciona como un *motor de inferencia simbólica*. El usuario envía una consulta lógica —por ejemplo, "¿es válido el contrato `contract2` ?"— y el servicio responde con el resultado que se *deriva automáticamente* a partir de los hechos y reglas declarados en la base de conocimiento. La respuesta no se calcula con un algoritmo imperativo escrito por el programador, sino que es *inferida* por el motor de resolución de Prolog.

El interés académico del proyecto, en el contexto de la materia de Teoría de Lenguajes de Programación, radica en que un único sistema funcional integra y hace dialogar **tres paradigmas de programación** distintos:

- **Programación lógica** — la base de conocimiento en Prolog.
- **Programación funcional** — la transformación de datos en JavaScript.
- **Programación asíncrona** — el modelo de ejecución de Node.js.

De esta manera, el proyecto muestra de forma concreta cómo paradigmas con filosofías diferentes — declarar relaciones frente a describir pasos— pueden combinarse dentro de una misma aplicación, comunicándose a través de una interfaz bien definida.

1.1. Objetivos

Objetivo general: desarrollar un sistema que demuestre la interacción entre distintos paradigmas de programación, evaluando consultas sobre una base de conocimiento y retornando resultados inferidos de manera automática.

Objetivos específicos:

- Modelar un dominio de conocimiento (contratos y clientes) mediante hechos y reglas declarativas en Prolog.
- Exponer el motor de inferencia mediante una API REST sencilla y clara.
- Procesar las consultas de forma asíncrona para atender múltiples solicitudes concurrentes sin bloquear el servidor.
- Devolver los resultados de la inferencia en un formato estándar (JSON).

2. Paradigmas de programación utilizados

El proyecto integra tres paradigmas. Cada uno cumple un rol específico y bien delimitado dentro del sistema.

2.1. Programación lógica (Prolog)

La programación lógica es un paradigma **declarativo**: el programador describe *qué* relaciones son verdaderas, no *cómo* calcularlas. El programa es un conjunto de **hechos** y **reglas**, y la ejecución consiste en demostrar si una consulta puede deducirse de ellos.

En este proyecto, el archivo `knowledge/base.pl` contiene la base de conocimiento. Por ejemplo:

```
% Hecho: contract1 tiene una penalidad aplicable
penalty_applicable(contract1).

% Regla: un contrato es válido si existe y NO tiene penalidad
valid_contract(X) :- contract(X), \+ penalty_applicable(X).
```

Los mecanismos propios de este paradigma que el sistema aprovecha son:

- **Unificación**: el motor empareja los términos de la consulta con los hechos y reglas de la base.
- **Backtracking**: ante varias alternativas, Prolog explora todas las soluciones posibles de forma automática.
- **Negación por fallo** (`\+`): una condición se considera falsa si no se puede demostrar que es verdadera.

El intérprete utilizado es **Tau Prolog**, una implementación de Prolog escrita en JavaScript, lo que permite ejecutar el motor lógico dentro del mismo proceso de Node.js.

2.2. Programación funcional (JavaScript)

La programación funcional trata las funciones como **valores de primera clase** y favorece la **transformación de datos** mediante funciones, evitando los efectos secundarios. En el módulo `src/inference.js` este paradigma se manifiesta en:

- **Funciones como valores**: a las operaciones de Tau Prolog (`consult` , `query` , `answers`) se les pasan funciones como argumentos (*callbacks*) que indican qué hacer ante el éxito o el error.
- **Transformación de datos**: cada solución cruda producida por el motor se convierte en una cadena legible aplicando la función `p1.format_answer(answer)` , y el conjunto de soluciones se acumula en un arreglo.
- **Composición**: la función `runQuery` se compone de pasos pequeños y bien definidos —leer, consultar, ejecutar, formatear— cada uno con una responsabilidad única.

```
// Transformación funcional: cada respuesta del motor
// se formatea y se agrega al arreglo de resultados
```

```
answers.push( p1.format_answer(answer) );
```

2.3. Programación asíncrona (Node.js)

Node.js se basa en un modelo de ejecución **asíncrono y no bloqueante** sobre un *event loop*. Las operaciones de entrada/salida no detienen el hilo principal, lo que permite atender varias solicitudes de forma concurrente.

En el proyecto este paradigma se observa en:

- **async/await** : la función `runQuery` es asíncrona y espera la finalización de cada etapa sin bloquear el servidor.
- **Promesas (Promise)**: las operaciones de Tau Prolog, basadas en *callbacks*, se "envuelven" en promesas para poder encadenarlas con `await`.
- **E/S no bloqueante**: la lectura de la base de conocimiento se realiza con `fs/promises`.
- **Concurrencia**: Express puede recibir y procesar varias peticiones al endpoint `/query` simultáneamente.

```
// Una API basada en callbacks se adapta al modelo de promesas
await new Promise((resolve, reject) => {
  session.consult(knowledgeBase, { success: resolve, error: reject });
});
```

Paradigma	Rol en el sistema	Archivo
Lógico	Define el conocimiento y realiza la inferencia	knowledge/base.pl
Funcional	Transforma y formatea los datos de la consulta	src/inference.js
Asíncrono	Coordina la E/S y atiende solicitudes concurrentes	src/server.js · src/inference.js

3. Arquitectura del sistema

El sistema se organiza en tres capas con responsabilidades claramente separadas. Esta separación facilita el mantenimiento: la base de conocimiento puede modificarse sin tocar el código del servidor.



3.1. Componentes

Componente	Tecnología	Responsabilidad
Servidor HTTP	Express	Expone el endpoint <code>/query</code> , recibe consultas Prolog y entrega resultados en JSON.
Motor de inferencia	Tau Prolog	Evalúa las reglas y hechos; realiza la inferencia lógica.
Base de conocimiento	Prolog	Define las relaciones lógicas y las condiciones de inferencia del dominio.
Capa asíncrona	Node.js (<code>async/await</code> , <code>Promise</code>)	Maneja la E/S no bloqueante y las solicitudes concurrentes.

3.2. Flujo de ejecución

Cuando un cliente envía una consulta, el sistema recorre los siguientes pasos:

1. El cliente envía un `POST` a `/query` con un cuerpo JSON que contiene la consulta lógica (ej. `penalty_applicable(contract1).`).
2. El servidor Express interpreta el cuerpo de la petición con el middleware `express.json()` y extrae el campo `query` .
3. Se invoca `runQuery` , que **lee de forma asíncrona** el archivo `base.pl` .
4. Se crea una sesión de Tau Prolog y se **carga** la base de conocimiento (`consult`).
5. El motor lógico **ejecuta la consulta** (`query`) y realiza la inferencia mediante unificación y backtracking.
6. Cada solución se **formatea** con `p1.format_answer` y se acumula en un arreglo.
7. El resultado se devuelve al cliente como una respuesta **JSON**.

La base de conocimiento se vuelve a cargar en cada consulta. Gracias a esto, cualquier cambio en `base.pl` tiene efecto inmediato sin necesidad de reiniciar el servidor.

4. Ejemplo de ejecución

A continuación se muestra una sesión real de uso del sistema. Primero se inicia el servidor y luego se envían varias consultas.

4.1. Inicio del servidor

```
$ npm start

Servidor corriendo en http://localhost:3000
```

4.2. Consulta de un hecho

Se pregunta si el contrato `contract1` tiene una penalidad aplicable. Como es un hecho declarado en la base, la inferencia es inmediata:

```
$ curl -X POST http://localhost:3000/query \
  -H "Content-Type: application/json" \
  -d '{"query":"penalty_applicable(contract1)."}'

{ "success": true, "result": ["true"] }
```

4.3. Consulta que aplica una regla

Se pregunta si `contract2` es un contrato válido. El motor evalúa la regla `valid_contract/1:contract2` existe y no tiene penalidad, por lo tanto es válido:

```
$ curl -X POST http://localhost:3000/query \
  -H "Content-Type: application/json" \
  -d '{"query":"valid_contract(contract2)."}'

{ "success": true, "result": ["true"] }
```

4.4. Consulta con variable (múltiples soluciones)

Al usar una variable (`x`), el motor emplea **backtracking** para encontrar *todos* los valores que satisfacen la consulta. Aquí se piden todos los contratos válidos:

```
$ curl -X POST http://localhost:3000/query \
  -H "Content-Type: application/json" \
  -d '{"query":"valid_contract(x)."}'

{ "success": true, "result": ["X = contract2", "X = contract3"] }
```

El resultado excluye `contract1`, ya que este tiene una penalidad y la regla lo descarta mediante la negación por fallo (`\+`).

4.5. Consulta de una regla encadenada

La regla `penalized_client/1` combina varios hechos: un cliente está penalizado si posee (`owns`) un contrato con penalidad. El motor encadena la inferencia automáticamente:

```
$ curl -X POST http://localhost:3000/query \  
  -H "Content-Type: application/json" \  
  -d '{"query":"penalized_client(X)."}'  
  
{ "success": true, "result": ["X = alice"] }
```

4.6. Consulta sin solución

Si la consulta no puede demostrarse a partir de la base de conocimiento, el sistema lo informa de forma explícita:

```
$ curl -X POST http://localhost:3000/query \  
  -H "Content-Type: application/json" \  
  -d '{"query":"penalty_applicable(contract2)."}'  
  
{ "success": false, "error": "No se encontró respuesta" }
```

Consulta	Tipo	Resultado
<code>penalty_applicable(contract1).</code>	Hecho	<code>["true"]</code>
<code>valid_contract(contract2).</code>	Regla	<code>["true"]</code>
<code>valid_contract(X).</code>	Regla con variable	<code>["X = contract2", "X = contract3"]</code>
<code>penalized_client(X).</code>	Regla encadenada	<code>["X = alice"]</code>
<code>penalty_applicable(contract2).</code>	Sin solución	<code>error</code>

5. Conclusiones y extensiones

5.1. Conclusiones

El proyecto cumplió su objetivo principal: construir un sistema funcional que demuestra, de manera concreta, la **interacción entre tres paradigmas de programación** dentro de una sola aplicación.

- La **programación lógica** demostró ser muy expresiva para modelar conocimiento: con pocas líneas de hechos y reglas se representa un dominio completo, y el motor deriva las respuestas sin que el programador escriba algoritmos de búsqueda.
- La **programación funcional** permitió transformar de forma clara y predecible los resultados del motor lógico hacia un formato apto para la web.
- La **programación asíncrona** de Node.js hizo posible integrar un motor basado en *callbacks* con un servidor web moderno, atendiendo solicitudes sin bloquear el proceso.
- La separación en capas (API, inferencia y conocimiento) resultó en un diseño ordenado, donde el dominio del problema se puede modificar editando únicamente el archivo `base.pl`.

En conjunto, el proyecto evidencia una idea central de la materia: cada paradigma ofrece herramientas adecuadas para distinto tipo de problemas, y un buen diseño puede combinarlos aprovechando las fortalezas de cada uno.

5.2. Extensiones posibles

El sistema es una base sólida sobre la que se pueden plantear varias mejoras:

Extensión	Descripción
Validación de consultas	Verificar la sintaxis de la consulta antes de ejecutarla y devolver mensajes de error más descriptivos.
Múltiples bases de conocimiento	Permitir seleccionar distintos archivos <code>.pl</code> según un parámetro de la petición.
Caché de la base	Cargar la base de conocimiento una sola vez en memoria para mejorar el rendimiento ante consultas frecuentes.
Interfaz web	Añadir un cliente con un formulario para escribir consultas y ver los resultados sin usar herramientas externas.
Carga dinámica de hechos	Exponer un endpoint que permita agregar hechos o reglas a la base en tiempo de ejecución.
Pruebas automatizadas	Incorporar un conjunto de pruebas que verifiquen las respuestas del endpoint <code>/query</code> para distintas consultas.